FLIGHTRESIST AI 2.0

QODER FINAL UPGRADE & COMPETITION EXECUTION PLAN

READ THIS FIRST

You are taking over an EXISTING, WORKING FlightResist AI 2.0 hackathon MVP.

This project has already been built and browser-verified.

DO NOT rebuild it from scratch.

DO NOT replace the existing architecture just because you prefer another stack.

DO NOT rewrite the UI unless a concrete judging, reliability, or integration problem requires it.

Your job is to transform the existing working MVP into the strongest possible competition version by progressively replacing simulated functionality with verified Atlas/Qoder capabilities while preserving the existing product, UX, deterministic engine, and fallback behavior.

The current project already contains a working:

- FlightResist operations cockpit
- deterministic trip-impact engine
- 42-candidate demo fixture
- 42 → 3 decision funnel
- deterministic optimization
- LLM explanation layer
- state machine
- human approval gate
- DemoProvider
- AtlasSandboxProvider abstraction
- SSE execution trace
- Prisma persistence
- recovery ledger
- run report
- browser-tested end-to-end demo

The current active environment is known to have used:

`[ENV: DETERMINISTIC DEMO]`

The immediate goal is to determine what REAL Atlas and Qoder capabilities are available now and upgrade the existing system without breaking the working demo.

NON-NEGOTIABLE RULES

Rule 1 — Preserve the working product

The current UI and core workflow are valuable assets.

Do not redesign the product from scratch.

The target flow remains:

```
NORMAL
  ↓
DISRUPTION DETECTED
  ↓
TRIP IMPACT ANALYSIS
  ↓
ATLAS / DEMO SEARCH
  ↓
DETERMINISTIC CONSTRAINT FILTERING
  ↓
MULTI-CRITERIA OPTIMIZATION
  ↓
AI EXPLANATION
  ↓
HUMAN APPROVAL
  ↓
PROVIDER EXECUTION
  ↓
RECOVERED
```

Rule 2 — Never invent Atlas capabilities

Before implementing or changing ANY Atlas operation:

1. Inspect the actual Atlas tools/Skill/API available in this Qoder environment.
2. Inspect the exact command/function/schema.
3. Verify the actual authentication mechanism.
4. Verify request and response formats.

5. Test the operation in Sandbox where possible.
6. Record the verified behavior.

Never infer an API from a README alone.

Never invent:

- CLI flags
- commands
- endpoints
- MCP tools
- schemas
- response fields
- payment semantics
- booking semantics
- ticketing semantics
- after-sales capabilities

If the capability is unavailable, use the existing DemoProvider fallback.

Rule 3 — Never fake real execution

REAL Atlas mode:

```
[ENV: ATLAS SANDBOX]
```

DEMO mode:

```
[ENV: DETERMINISTIC DEMO]
```

Never present:

- a synthetic PNR as a real PNR
- a simulated payment as a real payment
- a synthetic ticket as a real ticket
- a demo reference as a real airline booking

In DemoProvider mode use:

`demo_reference`

In real Atlas mode use provider-returned identifiers only.

Rule 4 — Keep one provider interface

All product logic must depend on:

```
BaseTravelProvider
```

Implementations:

```
AtlasSandboxProvider
DemoProvider
```

The application should be able to switch between:

```
AUTO
ATLAS_SANDBOX
DEMO
```

without changing the frontend or deterministic recovery engine.

Rule 5 — Deterministic engine remains authoritative

The LLM must NEVER override:

- hard constraints
- budget rules
- minimum connection time
- safety rules
- ranking arithmetic
- transaction authorization

The LLM explains deterministic results.

It does not invent them.

Rule 6 — Qoder integration must be genuine

Do not treat:

`qoder_mcp_config.json`

as evidence that Qoder was used.

If a Qoder agent/Skill/MCP/tool actually works, integrate it and document exactly what it does.

If something is unavailable, document that fact.

Never fabricate Qoder traces.

FINAL JUDGING OBJECTIVE

Optimize for:

Innovation — 30%

The core differentiator is:

> FlightResist does not merely notify travelers that a journey has failed. It computes how to recover the journey.

Feasibility — 30%

Show:

- deterministic safety logic
- real provider integration where available
- honest fallback
- typed architecture
- explicit approval
- observable execution

Qoder — 20%

Show genuine use of Qoder capabilities and tools where available.

Demo — 20%

The judge must understand the entire value proposition in under 3 minutes.

PHASE 0 — TAKEOVER AUDIT

Objective

Understand exactly what already exists before changing anything.

Inspect:

- repository structure
- current Git state
- package.json
- Next.js version
- Prisma schema
- environment configuration
- provider implementations
- engine implementation
- API routes
- SSE implementation
- frontend components
- existing README
- existing implementation status
- existing worklog
- existing tests
- browser verification artifacts
- dev server configuration

Do not delete anything.

Do not refactor anything yet.

Audit the existing working flow

Verify:

```
NORMAL
→ Trigger disruption
→ risk analysis
→ 42 candidates
→ 3 finalists
→ recommendation
→ approval
→ DemoProvider execution
→ RECOVERED
```

Run the complete flow once.

Record:

- current errors
- warnings
- latency
- broken states
- visual issues
- provider status
- current LLM provider
- current Atlas provider status

Required output

Create or update:

`QODER_UPGRADE_STATUS.md`

Include:

```
CURRENT SYSTEM
CURRENT PROVIDER
ATLAS STATUS
QODER STATUS
LLM STATUS
DATABASE STATUS
CURRENT TEST STATUS
CURRENT DEMO STATUS
RISKS
UPGRADE PLAN
```

Do NOT proceed to major modifications until this audit is complete.

PHASE 1 — ATLAS CAPABILITY DISCOVERY

Objective

Find out what Atlas capabilities are ACTUALLY available in Qoder.

Inspect:

- installed Atlas Skill
- CLI
- MCP tools
- environment variables
- credentials
- documentation available in the environment
- current Atlas sandbox access

Determine exactly whether these operations work:

```
Search flights
Fare verification
Order creation
Payment
Order retrieval
Ticketing status
Ancillaries
Post-booking operations, if actually available
```

Test the smallest valid Sandbox path.

Critical rule

The public/product specification is not authoritative if the installed environment behaves differently.

The real environment is authoritative.

Document:

```
SUPPORTED
UNSUPPORTED
UNVERIFIED
```

for every operation.

Phase 1 decision

Case A — Atlas fully works

Use:

```
AtlasSandboxProvider
```

as preferred provider.

Keep DemoProvider as fallback.

Case B — Atlas partially works

Integrate only the verified operations.

Use DemoProvider for missing portions.

Case C — Atlas unavailable

Keep DemoProvider active.

Do NOT fake Atlas.

Acceptance criteria

Phase 1 is complete only when:

- Atlas capability matrix is documented.
- Authentication status is known.
- At least one real Sandbox search has been tested if credentials permit.
- At least one real fare verification has been tested if supported.
- Any booking/payment operation actually available has been tested safely in Sandbox.
- Unsupported operations are explicitly documented.
- The application still works in DemoProvider mode.

PHASE 2 — QODER INTEGRATION

Objective

Make the project genuinely Qoder-compatible and, where capabilities exist, genuinely Qoder-powered.

Inspect the available:

- Agents
- Skills
- MCP

- model selection
- tool execution
- repository-aware coding features
- autonomous workflows

Do not create fake integrations.

Preferred architecture

The conceptual tooling boundary should be:

```
Qoder Agent
      ↓
Travel Tools / MCP / Skills
      ↓
FlightResist Provider Layer
      ↓
Atlas Sandbox
```

The recovery engine remains deterministic.

Qoder should provide the agent/tool orchestration layer, not replace the safety engine.

Create/document reusable Qoder capabilities where supported

Potential tool boundaries:

`search_flights`

Search provider inventory.

`verify_fare`

Verify current price/availability.

`create_order`

Create a supported Sandbox order.

`get_order_status`

Retrieve provider order state.

`get_ticketing_status`

Retrieve provider ticketing state where supported.

`analyze_recovery`

Feed deterministic recovery candidates into the reasoning workflow.

Do not expose dangerous side-effecting operations without explicit approval controls.

Acceptance criteria

Phase 2 is complete when:

- actual Qoder capabilities are documented;
- actual Qoder tools/Skills/MCP integrations are identified;
- no fake Qoder traces exist;
- at least one real Qoder-assisted workflow is demonstrated if the environment supports it;
- repository documentation explains exactly what Qoder contributes.

PHASE 3 — REPLACE THE DEMO PROVIDER WHERE POSSIBLE

Objective

Upgrade the existing DemoProvider system without changing the application architecture.

Keep:

```
BaseTravelProvider
```

and implement:

```
AtlasSandboxProvider
```

using ONLY verified Atlas capabilities.

Provider selection

Support:

```
AUTO
ATLAS_SANDBOX
DEMO
```

Behavior:

AUTO

Use Atlas if capability probe and authentication succeed.

Otherwise use DemoProvider.

ATLAS_SANDBOX

Use Atlas or return a clear configuration error.

Do not silently simulate.

DEMO

Always use deterministic fixture mode.

Frontend behavior

The same UI must work in both modes.

Display:

```
[ENV: ATLAS SANDBOX]
```

or:

```
[ENV: DETERMINISTIC DEMO]
```

Acceptance criteria

The existing demo must still work after Atlas integration.

Run:

```
Demo mode end-to-end
Atlas mode end-to-end if supported
```

and compare:

- state transitions
- candidate representation
- option cards
- approval flow
- execution state
- recovery ledger
- SSE trace

PHASE 4 — STRENGTHEN THE AGENTIC LAYER

Objective

Upgrade FlightResist from “deterministic application with LLM explanation” into a clearly agentic system without sacrificing deterministic safety.

Keep deterministic core

These remain normal code:

```
Constraint filtering
Scoring
Ranking
State machine
Transaction authorization
Provider adapter
```

Agentic responsibilities

Agents can handle:

Recovery Supervisor

Coordinates the workflow.

Impact Reasoner

Interprets downstream consequences.

Trade-Off Explainer

Explains why the recommended option wins.

Tool-Orchestration Agent

Invokes verified travel tools.

Do NOT create unnecessary agents merely for visual effect.

Each agent must have a concrete responsibility.

PHASE 5 — IMPROVE THE RECOVERY INTELLIGENCE

Objective

Make the product more differentiated than a standard rebooking chatbot.

The core concept should remain:

Trip Recovery Intelligence

When a disruption happens, show:

```
WHAT BROKE
      ↓
WHAT IT AFFECTS
      ↓
WHAT OPTIONS EXIST
      ↓
WHICH OPTIONS VIOLATE CONSTRAINTS
```

```
      ↓
WHICH OPTION BEST PRESERVES THE JOURNEY
```

Preserve the Trip Impact Graph

Keep:

- flight disruption
- connection impact
- arrival impact
- hotel impact
- transfer impact
- meeting/event impact

The graph must be deterministic and explainable.

Recovery ranking

Keep the multi-criteria optimization.

The engine must explain:

```
Why A lost
Why B won
Why C was not worth the additional cost
```

Improve the explanation

The AI explanation should reference actual deterministic facts:

```
+$43
+3h arrival
meeting preserved
within budget
safe connection
```

Never allow the LLM to invent facts.

PHASE 6 — TRUST, SAFETY, AND ENTERPRISE CREDIBILITY

This phase can increase Feasibility substantially.

Implement or verify:

Explicit approval

No execution without confirmation.

Idempotency

Double-clicking approval must NOT create duplicate bookings/orders.

Transaction state protection

Invalid state transitions return clear errors.

Provider failure handling

Examples:

```
Fare changed
Provider timeout
Payment failure
Order failure
Ticketing delay
```

Audit trail

Every important action should be traceable.

Secrets

No credentials in Git.

Acceptance criteria

Verify:

```
No approval → no transaction
Double approval → no duplicate transaction
Provider failure → FAILED state
Successful execution → RECOVERED state
```

PHASE 7 — PRODUCTION-ORIENTED HARDENING

Do not turn this into a giant enterprise platform.

Only improve areas that increase credibility or reliability.

Review:

- input validation
- error handling
- logging
- provider timeouts
- retries
- state recovery
- database consistency
- loading states
- accessibility
- responsive layout
- secrets management

Do NOT add:

- Kubernetes
- microservices
- enterprise IAM
- unnecessary infrastructure

PHASE 8 — DEMO MODE MUST REMAIN PERFECT

This phase is mandatory even after real Atlas integration.

The deterministic demo must always remain available.

The golden scenario:

```
NORMAL
↓
SIMULATE CANCELLATION
↓
87 RISK
↓
42 CANDIDATES
↓
3 SURVIVORS
↓
OPTION B RECOMMENDED
↓
AI EXPLANATION
↓
1-TAP APPROVAL
↓
EXECUTION
↓
RECOVERED
↓
RESIDUAL RISK 18
```

The demo must be reproducible from a clean start.

PHASE 9 — FINAL UI / DEMO POLISH

Do NOT redesign everything.

Polish only what increases Demo Presentation score.

Priorities:

1. First 20 seconds

A judge must immediately understand:

- what FlightResist is
- what broke
- why it matters

2. Risk transition

Make:

```
0 → 87
```

visually dramatic.

3. Decision funnel

Make:

```
42 → 30 → 12 → 3
```

extremely clear.

4. A/B/C comparison

The recommended option should be obvious.

5. Approval moment

Make:

```
CONFIRM RECOVERY
```

the central interaction.

6. Execution

Show actual provider/tool steps.

7. Recovery

Make the recovered state visually undeniable.

PHASE 10 — FINAL THREE-MINUTE DEMO

Build the final recording around ONE story.

0:00–0:20

Normal journey.

Voiceover:

> “FlightResist protects the journey, not just the booking.”

0:20–0:40

Cancellation occurs.

Risk:

```
87 / 100
CRITICAL
```

0:40–1:20

Agent analyzes.

Show:

```
42 candidates
→ constraints
→ 3 viable
```

1:20–1:50

Show A/B/C.

Explain:

> “Option B costs $43 more but preserves the critical meeting and stays within the traveler's budget.”

1:50–2:05

AI explanation.

2:05–2:20

Human approval.

2:20–2:45

Real Atlas Sandbox execution when available.

Otherwise:

clearly labeled DemoProvider execution.

2:45–3:00

Recovered journey.

Final statement:

> **“FlightResist doesn't tell travelers their journey is broken. It decides how to save it.”**

PHASE 11 — JUDGE EVIDENCE

Create:

`JUDGE_EVIDENCE.md`

It should contain a concise matrix:

Criterion	Evidence
Innovation	Trip Impact Graph + proactive autonomous recovery
Feasibility	Deterministic safety engine + Atlas provider + fallback
Qoder	Actual Qoder/Agent/MCP usage
Demo	End-to-end recovery workflow

Also document:

- what is real;
- what is Sandbox;
- what is deterministic demo;
- what is future roadmap.

Never claim simulated operations were real.

PHASE 12 — FINAL VALIDATION

Before declaring the project final:

Test 1 — Clean demo

Start from a clean state.

Test 2 — Golden path

Run the complete scenario.

Test 3 — Approval safety

Attempt execution without approval.

It must fail safely.

Test 4 — Duplicate approval

It must not double-execute.

Test 5 — Provider failure

It must become FAILED and recover gracefully.

Test 6 — Atlas

If available, execute the verified Sandbox path.

Test 7 — Demo fallback

Turn Atlas off and verify the demo still works.

Test 8 — Mobile

Verify the one-page cockpit.

Test 9 — Console

Zero critical frontend errors.

Test 10 — Build

Production build must succeed.

FINAL PHASE — FREEZE THE PROJECT

When all phases are complete:

DO NOT add random features.

Freeze the architecture.

Freeze the demo flow.

Freeze the judging narrative.

Create:

```
FINAL_STATUS.md
```

containing:

- final provider mode
- actual Atlas capabilities
- actual Qoder capabilities
- model used
- completed features
- known limitations
- demo instructions
- final test results
- final submission recommendation

MOST IMPORTANT PRIORITY ORDER

If time becomes limited, prioritize exactly this:

1. Real Atlas integration
2. Genuine Qoder integration
3. Deterministic recovery correctness
4. Reliable approval/execution flow
5. DemoProvider fallback

6. Demo polish
7. Documentation
8. Optional features

Never sacrifice 1–5 for optional features.

STOP CONDITIONS

STOP and report before making architectural changes if:

- Atlas behavior contradicts documentation;
- Qoder capability is unavailable;
- an implementation would require inventing an API;
- a provider operation would create an uncontrolled financial side effect;
- replacing an existing component would break the verified golden demo.

When blocked:

1. Document the blocker.
2. Preserve the current working functionality.
3. Use the smallest honest fallback.
4. Continue with independent phases.

FINAL COMMAND

Take ownership of the existing repository.

Do not rebuild FlightResist.

First execute PHASE 0.

Then proceed phase-by-phase.

At the end of EVERY phase:

1. verify the result;
2. document what changed;
3. run the relevant validation;
4. preserve the working golden demo;
5. continue to the next phase only if the current phase is stable.

The final goal is not “more features.”

The final goal is:

> **A genuinely working, technically credible, Atlas-connected, Qoder-integrated autonomous travel recovery product with an extraordinary three-minute demo.**